

COCO Ex2

Lennart Völz

April 26, 2026

2.1: An overview of non-uniform memory access

Primary contribution With scaling to more and more cores, UMA architectures fail to scale because of a single and shared memory bus that becomes the bottleneck. NUMA solves this by giving each processor (or socket) its own local memory and memory controller, which means memory BW scales with the number of cores and not against it. The tradeoff is, that remote accesses into other sockets lead to a latency penalty of roughly 50%. The article systematically reviews on how major operating systems (mostly Linux) handle NUMA in scheduling, page placement, etc.

Key insight The fundamental key insight is that specifying memory policies do not cause any allocation in hardware, i.e. pages that have not been touched (e.g. uninitialized variables without default allocator) also do not have physical memory attached to them. This means that whichever NUMA region first accesses that page, determines the placement in memory (for operating systems this could mean memory layout is determined by boot-time processes). Two of the common policies are node local, where the allocation occurs from the memory node local the process execution, and interleaved, where allocation occurs round-robin.

My thoughts The article frames how important NUMA aware placement and access is when using server hardware. Despite reviewing the concept from the operating system side, it becomes clear that this also applies for running multi-threaded algorithms efficiently. Because the article was published in 2013, the Linux kernel was not NUMA aware at this time. But not as the author prognosticated, automatic NUMA balancing is now a standard feature. The operating system can physically move pages in memory.

2.2: Parallel Pointer Chasing Benchmark

The parallel implementation launches threads, that each independently traverse the same shared array via pointer chasing. Threads are not partitioning the work, they duplicate it.

Performance Analysis Under the MESI cache protocol, a cache line can be held by multiple cores without any invalidation. Since data is only read, there is no occurrence of false sharing. This is also what we can see in Figure 1. Adding threads does not increase latency for read-only pointer chasing. The memory hierarchy allows multiple concurrent

readers.

In both cases, the latency is relatively low for small arrays that fit in L1/L2 cache (also with small strides). For these cases, the spatial locality is high, thus latency is low. For larger arrays and larger strides (greater than 64 bytes), every pointer hop fetches a new cache line load from memory.

I think there might be an error in the exercise because reads cannot cause false sharing I believe? To actually observe false sharing, the code would need threads that write to a shared unpadding array. Then to fix the false sharing, use alignment of the cache line size.

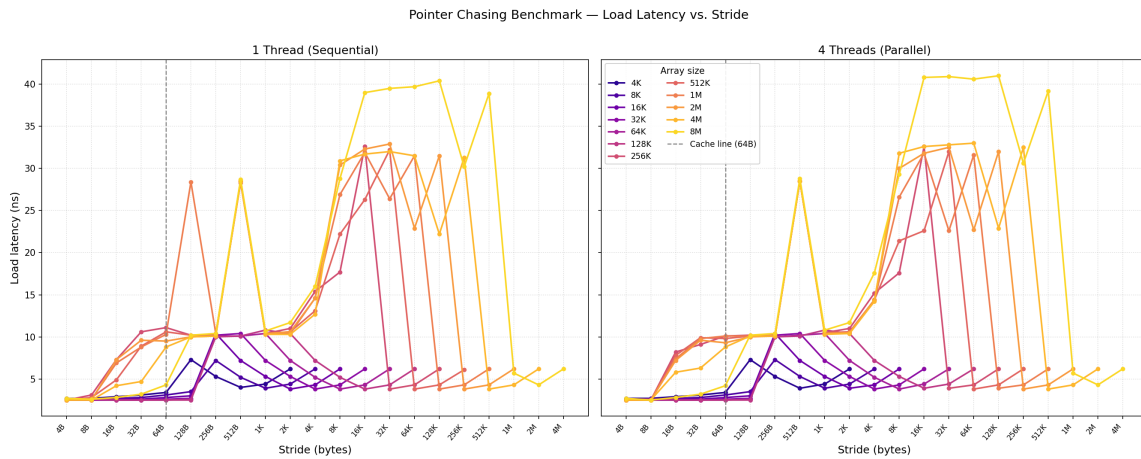


Figure 1: Comparison plot of the parallel pointer chasing benchmark: single threaded against running with four threads.

2.3: Multi-threaded Load Bandwidth with Inference Engine Considerations

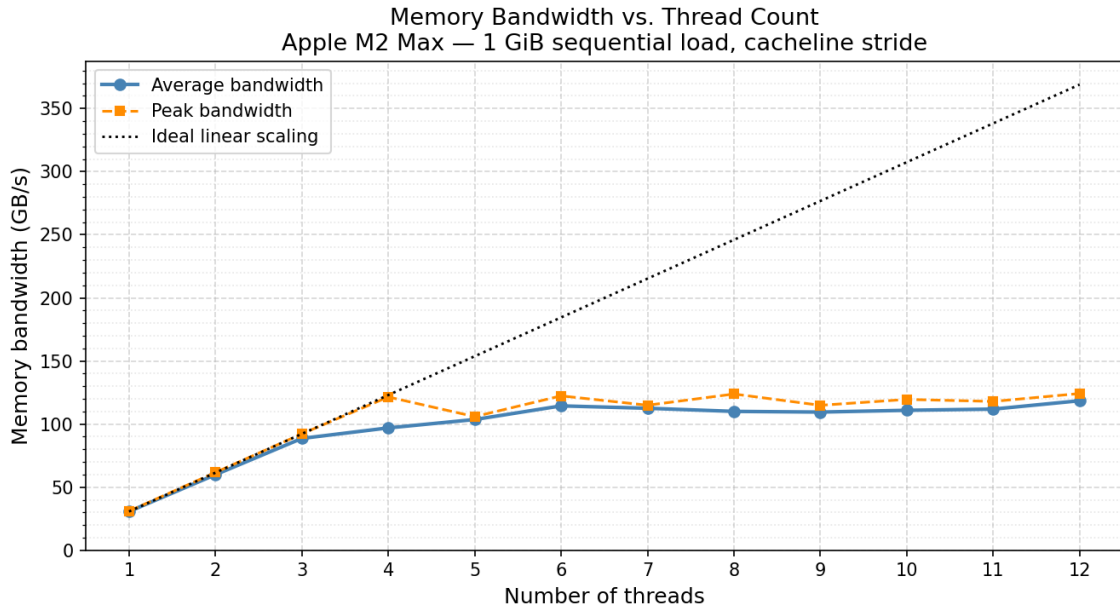


Figure 2: Load bandwidth scaling with number of threads.

The benchmark shows that BW scales almost linearly for 1-3 threads. Then the slope gets weaker for 4-6 threads, and completely flattens out afterwards, which means that the bottleneck is the memory interconnect.

Connection to inference engines Autoregressive LLMs are memory bound for decoding tasks, because for each output token the entire model weight matrix has to be streamed once. A single thread captures only limited possible BW (on my Apple M2 Max roughly 25%). To reach peak BW, multiple threads are needed. For server CPUs, also NUMA effects should be considered.

The first thing to speed up inference tasks is launching a thread pool and reusing threads, because spawning threads is really expensive in the hot path. Because loading the weights (or also storing for KV-cache) is making inference memory bound, quantizing the weights to e.g. INT8 or INT4 leads to much higher tokens/s due to higher possible throughput.